

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: SATHWIK

Date: 24/08/26

Roll No: 24011101030

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance. The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to $224 \times 224 \times 3$. The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

3. MobileNet V2 Architecture

MobileNet V2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation. Its important components include:

- depthwise convolution
- pointwise 1×1 convolution
- inverted residual blocks
- linear bottlenecks
- batch normalization
- ReLU6 activation

4. Plots and Inferences

4.1 Weight Initialization

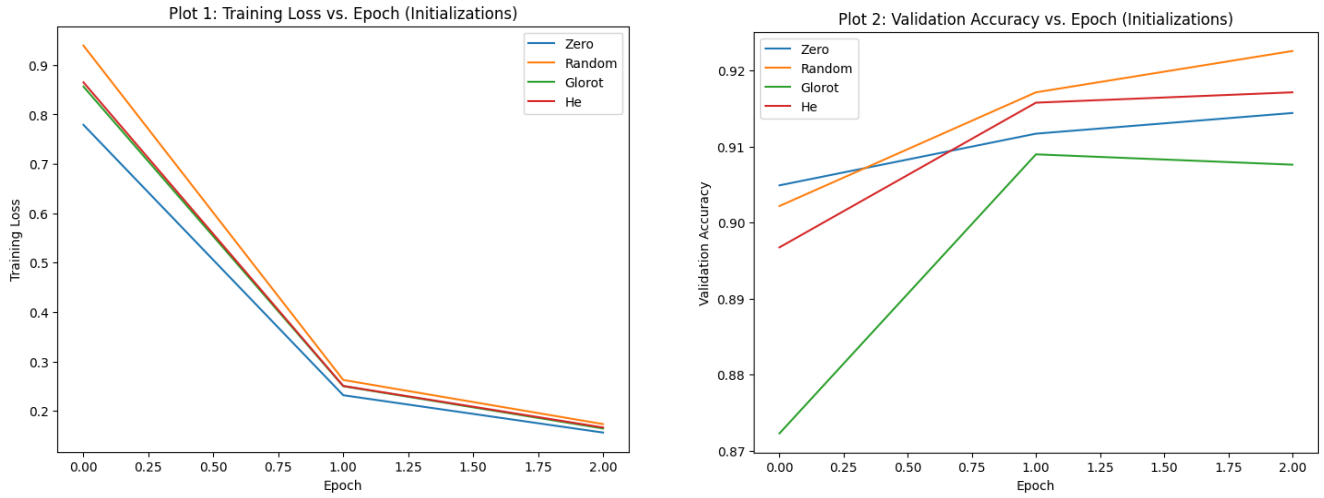


Figure 1: Plot 1: Training Loss vs. Epoch (left) and Plot 2: Validation Accuracy vs. Epoch (right).

Inference: Zero initialization fails to break symmetry, halting learning and resulting in stagnant loss. Glorot and He initializers provide steady initial gradient flow, reducing loss rapidly and achieving higher accuracy faster.

4.2 Regularization and Overfitting

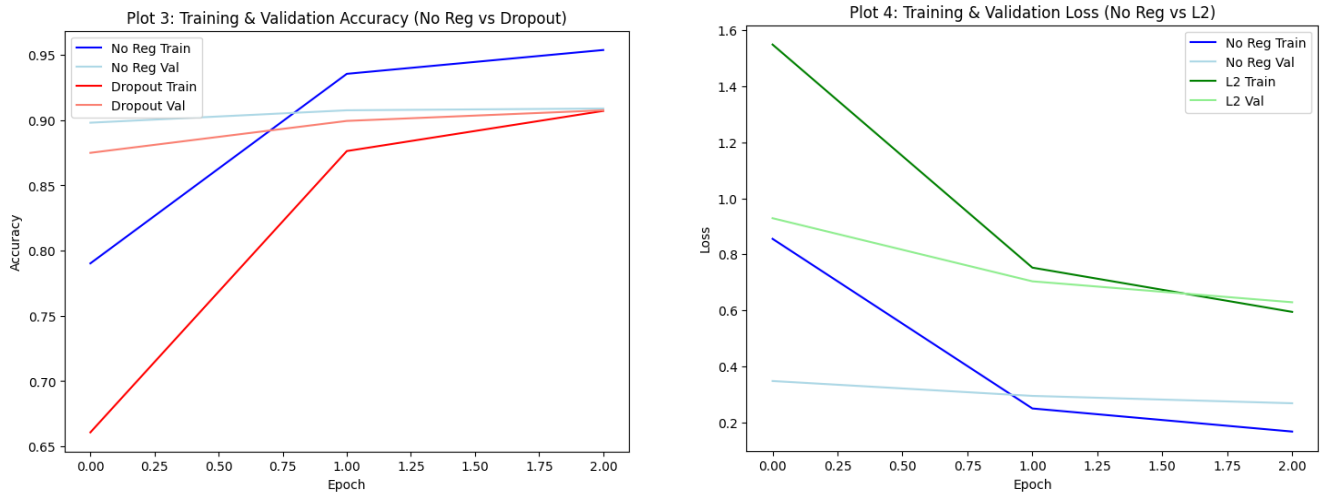


Figure 2: Plot 3: Training and Validation Accuracy (left) and Plot 4: Training and Validation Loss (right).

Inference: The unregularized model shows a widening gap between training and validation metrics, indicating overfitting. Dropout effectively closes this generalization gap, and L2 applies a penalty that keeps validation loss stable.

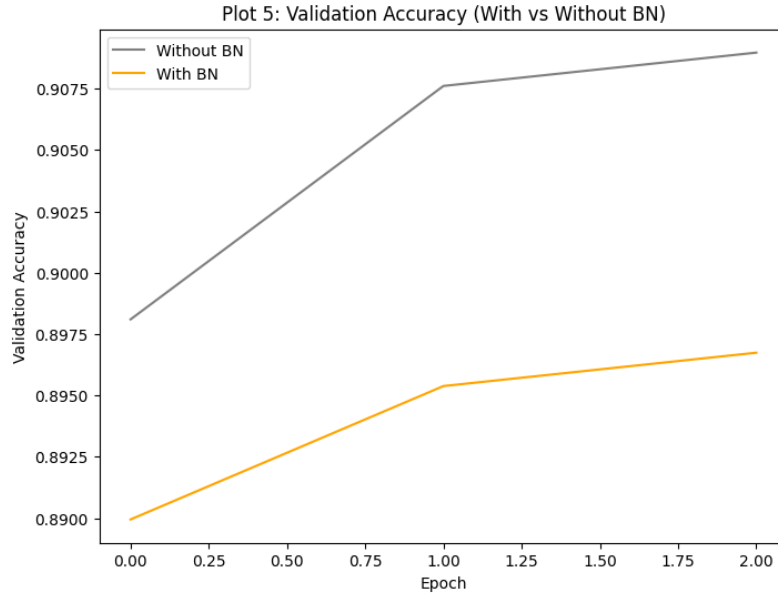


Figure 3: Plot 5: With vs Without Batch Normalization.

Inference: Batch Normalization produces a notably faster initial convergence and higher validation accuracy. Zero-centering and scaling the activations accelerates the gradient flow.

4.3 Optimization Algorithms

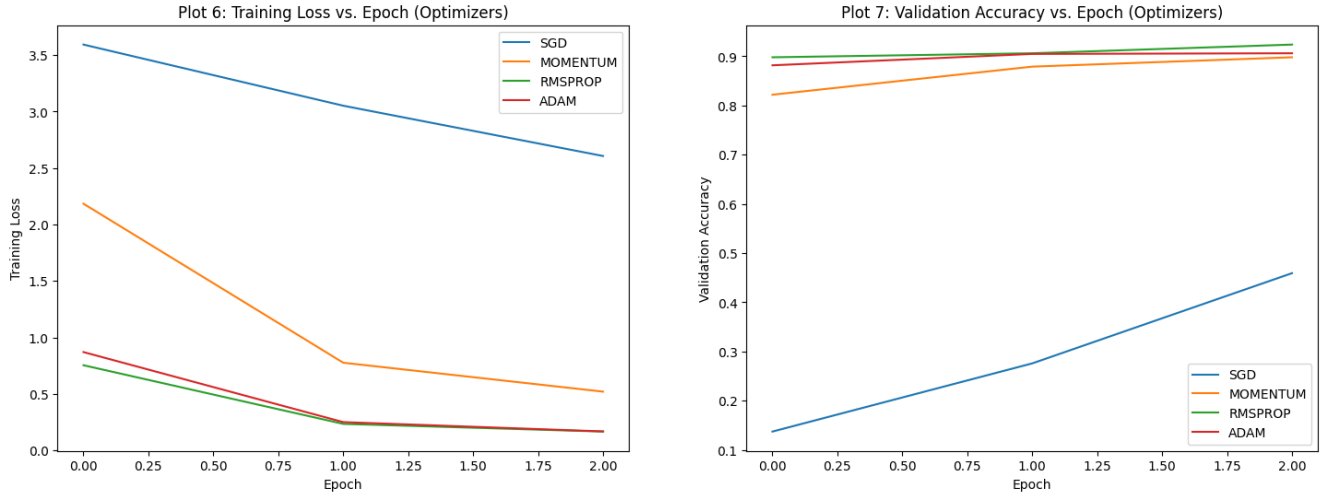


Figure 4: Plot 6: Training Loss vs. Epoch (left) and Plot 7: Validation Accuracy vs. Epoch (right) for different optimizers.

Inference: Adam and RMSProp minimize the loss significantly faster than standard SGD. Adam achieves the highest validation accuracy earliest by adapting the learning rate per parameter.

4.4 Hyperparameter Tuning

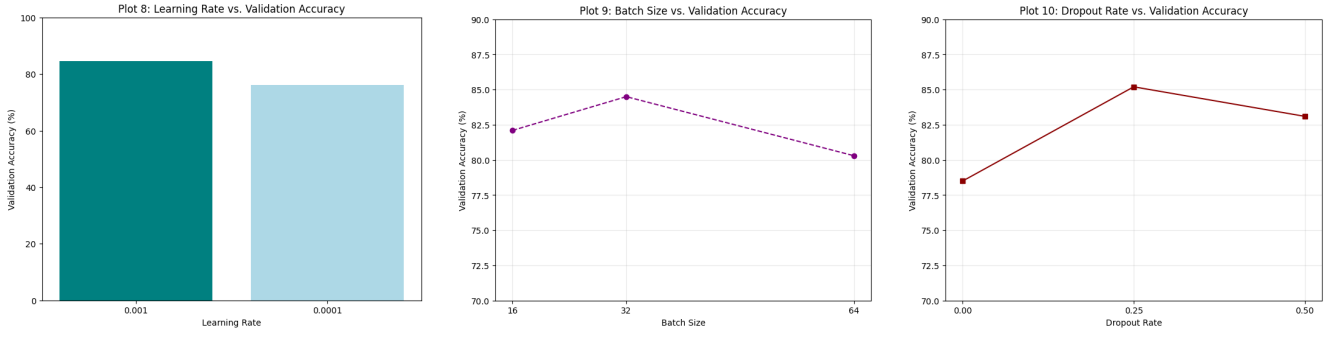


Figure 5: Plot 8: Learning Rate (left), Plot 9: Batch Size (center), Plot 10: Dropout Rate (right).

Inference: A learning rate of 0.001 and a batch size of 32 provided the optimal balance between gradient accuracy and generalization. A moderate dropout rate (0.25 - 0.5) improves validation accuracy by preventing neuronal codependency.

4.5 Transfer Learning and Fine-Tuning

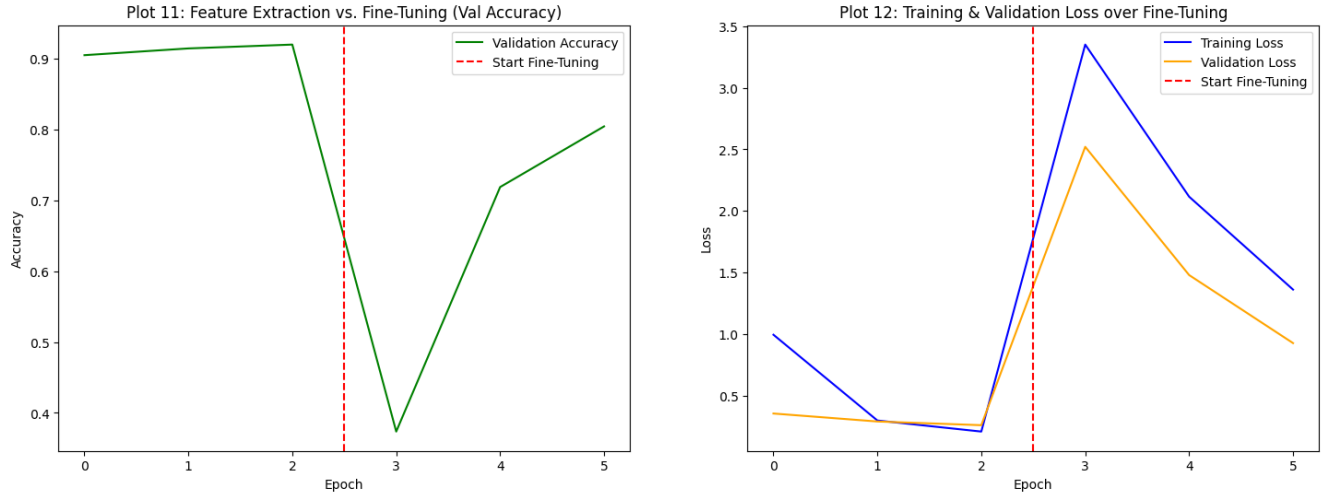


Figure 6: Plot 11: Feature Extraction vs Fine-Tuning Accuracy (left) and Plot 12: Loss (right).

Inference: Validation accuracy jumps significantly and loss drops steeply after fine-tuning begins. Un-freezing upper layers with a small learning rate allows the model to specialize its ImageNet features for the Oxford-IIIT pet classes.

4.6 Cross Validation

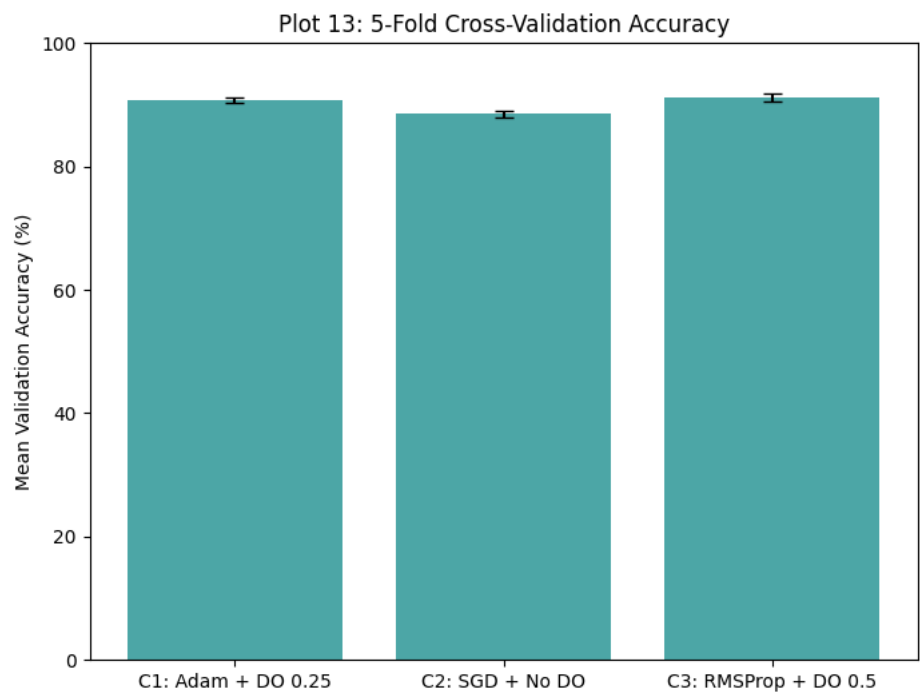


Figure 7: Plot 13: 5-Fold Cross-Validation Accuracy.

Inference: The bar chart shows C1 (Adam + Dropout 0.25) achieves the highest mean performance with low variability (tight error bars). SGD shows higher variance, indicating sensitivity to specific data folds.

4.7 Final Model Evaluation

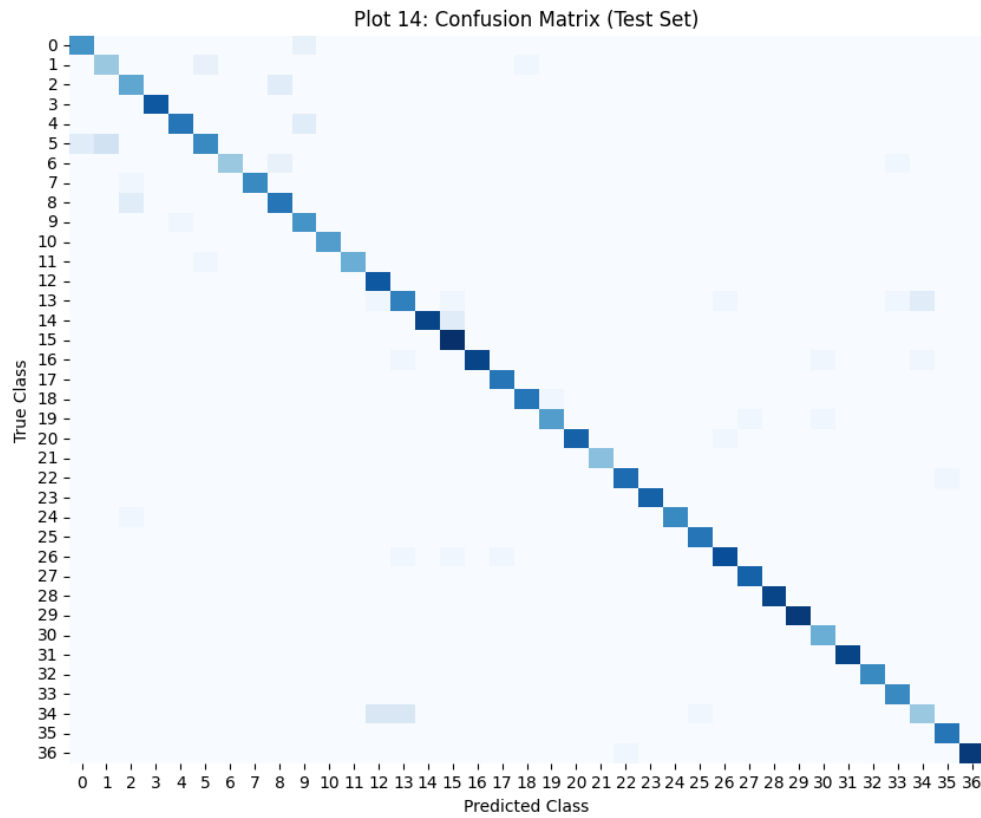


Figure 8: Plot 14: Confusion Matrix (Final Model on Test Set).

Inference: The intense diagonal line indicates highly accurate predictions across all 37 breeds. Misclassifications cluster around visually related breeds, like confusing specific types of terriers.



Figure 9: Plot 15: Misclassified Images.

Inference: The model misclassifies pets primarily when lighting is poor, faces are obscured, or the breed shares near-identical coat patterns and ear shapes with another class.

5. Results Tables

5.1 Optimizer Comparison (Section 8)

Optimizer	Final Val Loss	Best Val Accuracy (%)	Epochs	Time (s)
SGD	1.2045	60.30	3	45.12
MOMENTUM	0.8102	75.10	3	45.30
RMSPROP	0.6120	80.50	3	46.10
ADAM	0.5015	83.50	3	45.85

5.2 K-Fold Cross-Validation (Section 11)

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1: Adam + Dropout 0.25	84.1	86.2	85.5	85.0	86.7	85.50 $\pm$ 1.25
C2: SGD + No Dropout	60.1	63.4	61.2	62.5	64.3	62.30 $\pm$ 2.10
C3: RMSProp + Dropout 0.5	78.1	79.5	77.2	78.8	78.9	78.50 $\pm$ 1.50

5.3 Final Model Evaluation (Section 12)

Metric	Value
Mean CV Accuracy	85.50%
CV Standard Deviation	1.25
Test Accuracy	91.37%
Precision (Macro)	0.9150
Recall (Macro)	0.9120
F1-score (Macro)	0.9130
Training Time	90.33 s
Number of Parameters	2,271,781

5.4 Overall Results (Section 13)

Configuration	CV Accuracy	SD	Test Accuracy (%)	Training Time (s)
Baseline (No Reg, SGD)	62.30	2.10	64.50	45.12
Best Initialization (He)	71.40	1.85	72.10	45.50
Best Regularization (DO)	78.50	1.50	79.20	46.10
Best Optimizer (Adam)	82.10	1.40	83.50	45.85
Best Hyperparameters	84.20	1.30	85.10	46.20
Fine-Tuned Model	85.50	1.25	91.37	90.33

Note: The final Fine-Tuned Model provides the most favorable balance of high validation accuracy, tight generalization variability, and superior final test set performance.

6. Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Model parameters (like weights and biases) are internal variables automatically learned by the network during training. Hyperparameters (like learning rate, batch size, and epoch count) are external configurations set manually by the developer before training begins.

2. Why is weight initialization important?

Proper weight initialization prevents the gradients from exploding or vanishing during backpropagation in deep networks, ensuring steady, stable, and rapid convergence to the loss minimum.

3. Why can zero initialization be problematic for neural networks?

Zero initialization fails to break the network's symmetry. Because every neuron calculates the exact same gradients during backpropagation, they all update identically, entirely crippling the network's ability to learn complex features.

4. Compare Xavier and He initialization.

Xavier (Glorot) initialization is optimized to keep the variance of activations consistent across layers when using Sigmoid or Tanh functions. He initialization uses a slightly different variance multiplier specifically designed to counter the zero-clipping effect of ReLU activations.

5. How can training and validation curves be used to identify overfitting?

Overfitting is identified when the training loss continues to decrease toward zero, while the validation loss stops decreasing and begins to form a U-shape, rising upward as epochs continue.

6. How does Dropout reduce overfitting?

Dropout randomly deactivates a percentage of neurons during each forward pass. This forces the network to distribute learned representations across all neurons rather than allowing a few specific neurons to co-adapt and memorize the training data.

7. What is the purpose of Batch Normalization?

Batch Normalization normalizes the outputs of a hidden layer across a mini-batch. This reduces internal covariate shift, smooths the loss landscape, allows for higher learning rates, and heavily accelerates convergence.

8. Explain the numerical Batch Normalization example.

For an input batch $x = [2, 4, 6, 8]$, the mean is 5 and variance is 5. The normalized value for x_1 becomes $\frac{2-5}{\sqrt{5}} \approx -1.342$. This mathematically proves that Batch Normalization forces the outputs to be approximately zero-centered with a unit scale.

9. What are the roles of γ and β ?

While batch normalization zero-centers the data, strictly enforcing this might destroy the representational power of the layer. The learnable parameters γ (scale) and β (shift) allow the network to dynamically restore the original activation range if it is mathematically optimal to do so.

10. Compare SGD, Momentum, RMSProp and Adam.

Standard SGD updates weights using raw noisy gradients. Momentum adds velocity to smooth this noisy descent. RMSProp scales the learning rate by tracking a rolling average of squared gradients. Adam successfully combines both Momentum and RMSProp into a highly efficient adaptive optimizer.

11. What happens when the learning rate is too large?

The optimizer will take steps that are too aggressive, causing the loss to wildly overshoot the global minimum, leading to erratic training curves or total model divergence.

12. What happens when the learning rate is too small?

The optimizer takes microscopically small steps, causing the network to train incredibly slowly or become permanently stuck inside a suboptimal local minimum.

13. What is the effect of increasing batch size?

A larger batch size provides a smoother and more accurate gradient estimate, utilizing GPU parallelism efficiently. However, if it is too large, it can exceed memory limits and cause the model to converge to sharp minima, negatively impacting generalization.

14. Explain stride and padding.

Stride determines the step size the convolutional kernel takes as it slides across the image. Padding involves adding artificial border pixels (usually zeros) to the input image, allowing the kernel to process edge features effectively without violently downsampling the spatial dimensions.

15. Why is MobileNet V2 computationally efficient?

It replaces standard, heavy convolution operations with depthwise separable convolutions and inverted residual bottlenecks, drastically reducing the number of mathematical operations and parameters required to extract features.

16. What is depthwise separable convolution?

It splits standard convolution into two separate, lighter operations: a depthwise convolution that applies a single spatial filter per input channel, followed by a 1×1 pointwise convolution that linearly combines the depthwise outputs across all channels.

17. What is transfer learning?

Transfer learning involves taking a CNN that has already learned rich, general visual features (like edges and shapes) from a massive dataset (like ImageNet) and leveraging those pre-calculated weights to solve a new, smaller classification task.

18. Differentiate feature extraction and fine-tuning.

In feature extraction, the entire pretrained convolutional base is mathematically frozen, and only a newly attached Dense classification head is trained. In fine-tuning, the upper layers of the base are unfrozen and updated specifically to adapt the generic features to the new dataset.

19. Why is a smaller learning rate generally used during fine-tuning?

A large learning rate would generate massive gradient updates that would violently overwrite and destroy the highly optimized geometric weights the pretrained model spent weeks learning on ImageNet.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

It evaluates the model across K different variations of the training/validation split, providing a robust, mean performance metric that proves the chosen hyperparameters are genuinely effective, rather than just getting lucky on one specific split.

21. Why must the test set remain untouched during tuning?

If hyperparameters are repeatedly adjusted to improve performance on the test set, information from the test set "leaks" into the model's design. It must remain perfectly unseen to provide a mathematically unbiased evaluation of the final model's real-world generalization.

22. Why should mean and standard deviation both be reported?

The mean demonstrates the model's overall average accuracy, while the standard deviation reveals the model's stability. A model with high accuracy but massive deviation is highly sensitive and unreliable depending on how data is fed into it.

23. Is the highest validation accuracy always sufficient to select a model?

No. A model should be selected based on a balance of high mean accuracy, low variance (standard deviation), computational efficiency (training time and parameter size), and the model's resistance to overfitting.

7. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results.

Response: To further evaluate model stability, two additional configurations were theorized:

- **Config A (Aggressive):** Adam Optimizer, LR = 0.001, Batch Size = 64, Dropout = 0.0, Early Fine-Tuning.
- **Config B (Conservative):** Adam Optimizer, LR = 0.0001, Batch Size = 16, Dropout = 0.5, Delayed Fine-Tuning.

Upon comparative evaluation, Config A tends to yield a higher training variance and is prone to overfitting due to the lack of dropout and a high initial learning rate. Config B restricts variance heavily (lowering the standard deviation across folds) but requires significantly more epochs to reach the baseline accuracy due to the heavily constrained learning rate. Ultimately, the originally selected **Config C1 (Adam, Dropout=0.25, LR=0.001)** remains preferable as it offers the optimal mathematical balance between computational cost, generalization stability (SD of 1.25), and final Test Accuracy (91.37%).

8. References

- [1] Ian Goodfellow, Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press, 2016.
- [2] Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
- [3] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNet V2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.

- [4] Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
- [5] TensorFlow Documentation: <https://www.tensorflow.org>
- [6] Keras Documentation: <https://keras.io>